

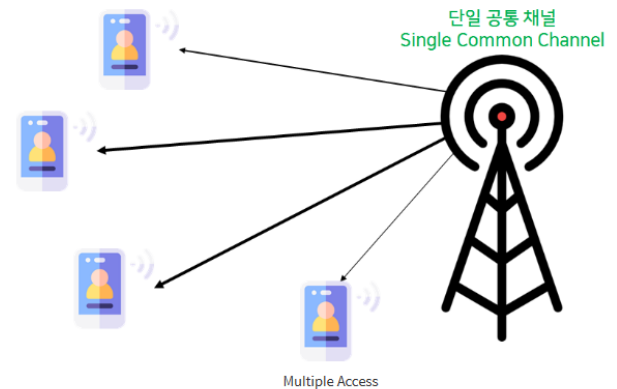
Practice 2

CSMA simulation in ns-3

Sangheon Pack
(shpack@korea.ac.kr)
2023.09.21

Multiple Access Channel

- A communication channel that allows multiple users or devices to send and receive data at the same time
- Conflicts between multiple users can occur
- Various multiple access channel (MAC) Protocols Exist
 - **Carrier Sense Multiple Access (CSMA)**
 - Frequency Division Multiple Access (FDMA)
 - Code Division Multiple Access (CDMA)
 - Time Division Multiple Access (TDMA)

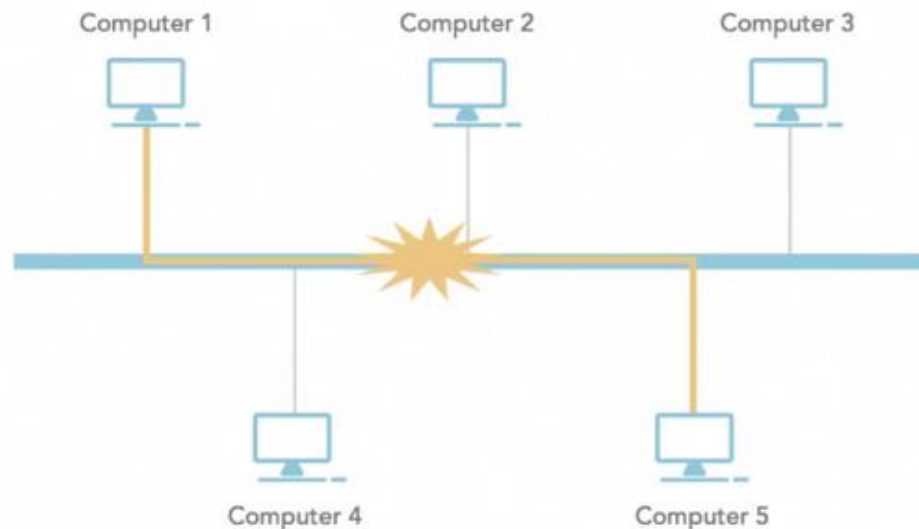


CSMA

- **Random Access** (Contention Based Access)
 - ✓ No node has a higher priority than any other node, and no other node can be controlled
- Mainly used in **Ethernet networks** and **wireless LAN (Wi-Fi)**
- When a host shares one medium/link, a **conflict may** occur if two or more hosts use the link at the same time

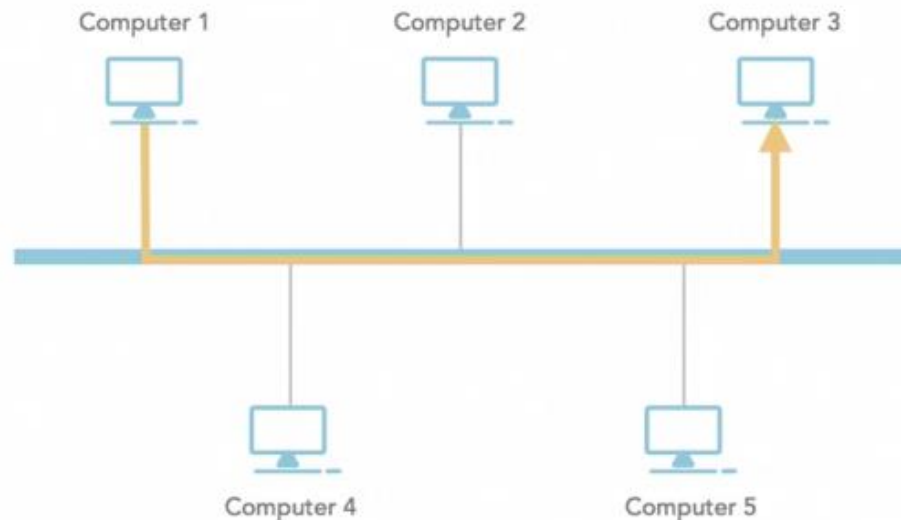
CSMA/CD (Collision Detection)

- Using on **wired Ethernet networks**
- To check if the channel is empty and transmit a signal before transmitting
- If there is a conflict, wait for some time and resend accordingly



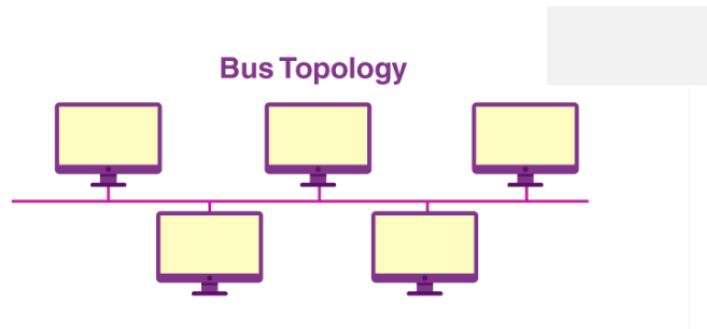
CSMA/CA (Collision Avoidance)

- Using on **wireless Ethernet networks**
- Trying to **avoid collision situations** as much as possible
- Even if the channel is empty, the transmission proceeds after a few seconds, or **additional packets** are transmitted to prevent a collision



CSMA Model in ns-3

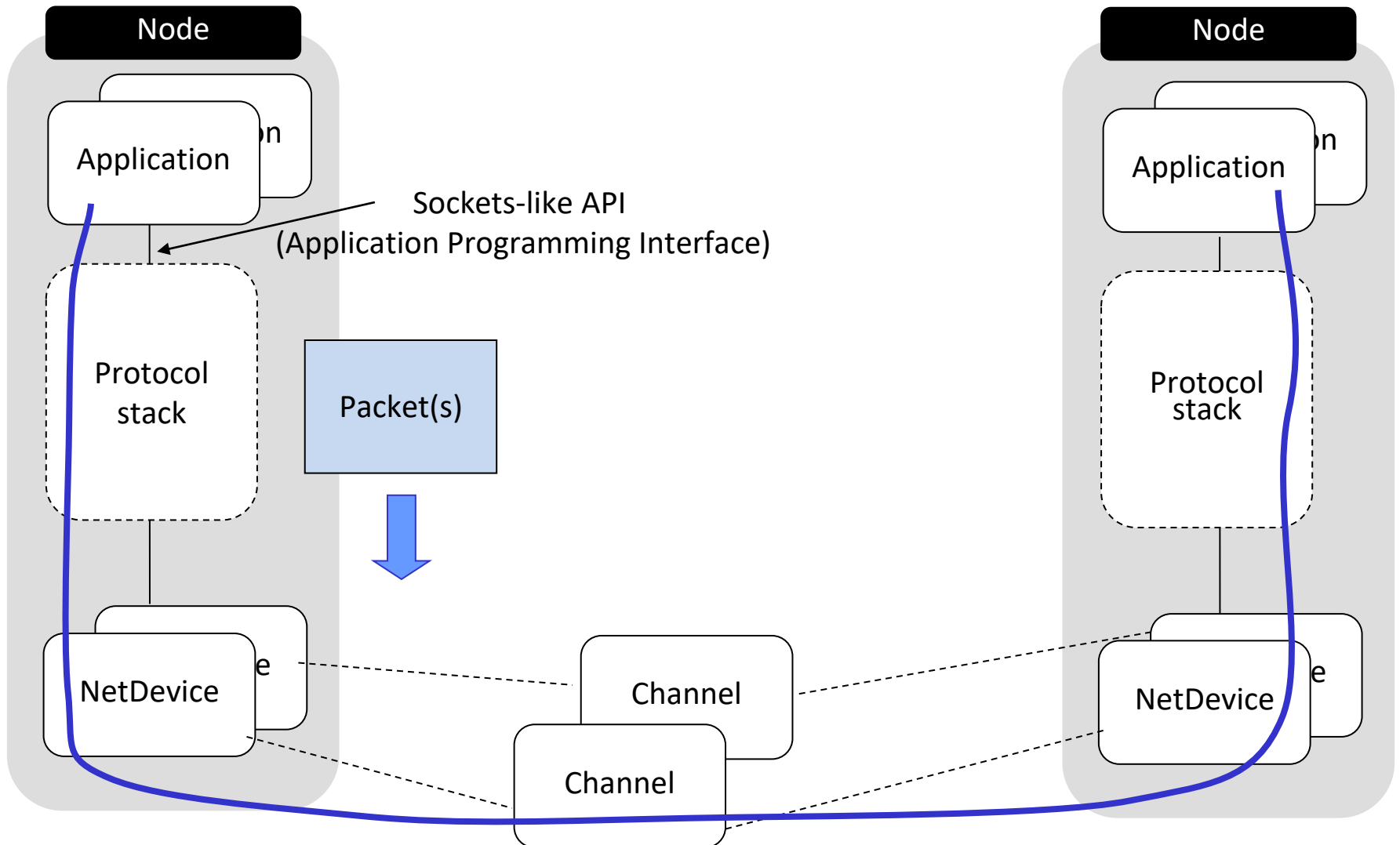
- The NS-3 CSMA device models a simple bus network in the spirit of Ethernet
- The NS-3 CSMA device does not model collision detection, and any transmission in progress is not “jammed”
- CSMA channel class
 - ✓ A single busy flag is used to indicate if the channel is currently in use
 - ✓ It does not take into account the distances between stations or the speed of light to determine collisions



CSMA Channel Model

- No limit for the number of devices connected to the channel
- Attributes
 - ✓ *DataRate* and *Delay*
- Three states
 - ✓ IDLE, TRANSMITTING, and PROPAGATING
 - ✓ IDLE -> TRANSMITTING -> PROPAGATING -> IDLE

ns-3 Basic Simulation Model



Creating Network Topology (1/3)

- *NodeContainer*
 - ✓ *NodeContainer Ptr<Node>*
 - ✓ *NodeContainer::Create (n)*
(n: Number of Nodes)
- Example (*myfirst.cc*)
 - NodeContainer nodes;
 - nodes.Create (2);

Creating Network Topology (2/3)

- *PointToPointHelper*
 - ✓ *void PointToPointHelper::SetDeviceAttribute (device attribute, value)*
 - ✓ *void PointToPointHelper::SetChannelAttribute (channel attribute, value)*
 - ✓ *NetDeviceContainer PointToPointHelper::Install (NodeContainer c)*
- *NetDeviceContainer*
 - ✓ *NetDeviceContainer Ptr<NetDevice>*
- Example (*myfirst.cc*)

```
PointToPointHelper pointToPoint;  
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));  
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));  
  
NetDeviceContainer devices;  
devices = pointToPoint.Install (nodes);
```

Creating Network Topology (3/3)

- *InternetStackHelper*
 - ✓ *InternetStackHelper::install (NodeContainer c)*
- *Ipv4AddressHelper*
 - ✓ *void Ipv4AddressHelper::SetBase (ipv4 address, ipv4 mask)*
 - ✓ *void Ipv4AddressHelper::Assign (NetDeviceContainer c)*
- *Ipv4InterfaceContainer*
- Example (myfirst.cc)

```
InternetStackHelper stack;  
stack.Install (nodes);  
  
Ipv4AddressHelper address;  
address.SetBase ("10.1.1.0", "255.255.255.0");  
Ipv4InterfaceContainer interfaces = address.Assign (devices);
```

Creating Application (1/2)

- *UdpEchoServerHelper*
 - ✓ *UdpEchoServerHelper::UdpEchoServerHelper (port)*
 - ✓ *ApplicationContainer UdpEchoServerHelper::Install (NodeContainer c)*
- *UdpEchoClientHelper*
 - ✓ *UdpEchoClientHelper::UdpEchoClientHelper (ip, port)*
 - ✓ *void UdpEchoClientHelper::Setattribute (name, value)*
 - ✓ *ApplicationContainer UdpEchoClientHelper::Install (NodeContainer c)*
- *ApplicationContainer*
 - ✓ *void ApplicationContainer::Start (Seconds(start time))*
 - ✓ *void ApplicationContainer::Stop (Seconds(end time))*

Creating Application (2/2)

- Example (*myfirst.cc*)

```
UdpEchoServerHelper echoServer (9);
```

```
ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));
```

```
serverApps.Start (Seconds (1.0));
```

```
serverApps.Stop (Seconds (10.0));
```

```
UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);
```

```
echoClient.SetAttribute ("MaxPackets", UIntegerValue (1));
```

```
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
```

```
echoClient.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));
```

```
clientApps.Start (Seconds (2.0));
```

```
clientApps.Stop (Seconds (10.0));
```

Running Simulator

- Start Simulator
 - ✓ *Simulator::Run ();*
- Destroy Simulator
 - ✓ *Simulator::Destroy ();*

CsmaHelper (1/3)

- *void csmaHelper::SetChannelAttribute (channel attribute, value)*
 - ✓ Set the channel attributes
- Channel Attribute parameters
 - ✓ **DataRate**: The transmission data rate to be provided to devices connected to the channel
 - ✓ **Delay**: Propagation delay through the channel
- Example (*mysecond.cc*)

```
CsmaHelper csma;  
csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));  
csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));
```

CsmaHelper (2/3)

- *void csmaHelper::SetDeviceAttribute (device attribute, value)*
 - ✓ Set the attributes of *csmaNetDevice*
- Device Attribute parameters
 - ✓ **Address**: The MAC address of this device
 - ✓ **Mtu**: The MAC-level Maximum Transmission Unit
 - ✓ **EncapsulationMode**: The link-layer encapsulation type to use
 - ✓ **SendEnable**: Enable or disable the transmitter section of the device
 - ✓ **ReceiveEnable**: Enable or disable the receiver section of the device
 - ✓ **ReceiveErrorModel, TxQueue**

CsmaHelper (3/3)

- *NetDeviceContainer csmaHelper::Install (NodeContainer c)*
 - ✓ Install network devices on csma nodes (in *NodeContainer*)
 - ✓ return *NetDeviceContainer* which holds the installed devices
- Example (*mysecond.cc*)

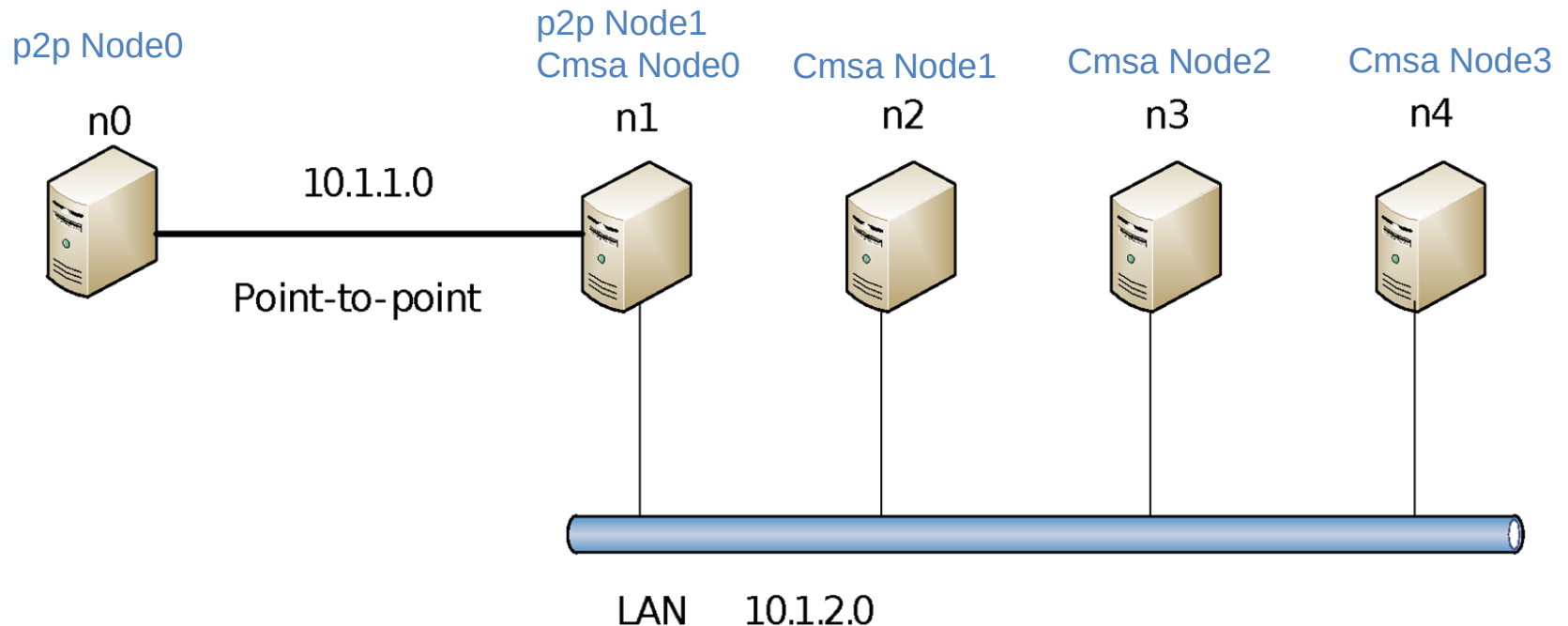
```
NetDeviceContainer csmaDevices;  
csmaDevices = csma.Install (csmaNodes);
```

Simulation Example

- Copy the example (*second.cc*) into your scratch folder (*mysecond.cc*)
 - ✓ `../ns-3.35$ cp examples/tutorial/second.cc scratch/mysecond.cc`
- Run
 - ✓ `../ns-3.35$ ./waf --run scratch/mysecond`
- Edit your source code
 - ✓ `../ns-3.35$ gedit scratch/mysecond.cc`

CSMA Topology

- CSMA network topology examples (*mysecond.cc*)

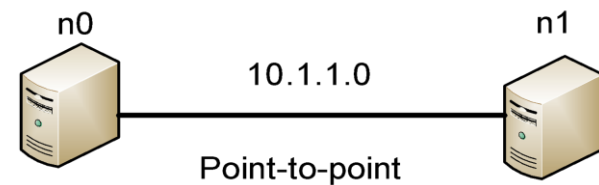


CSMA Node Creation

- Example (*mysecond.cc*)

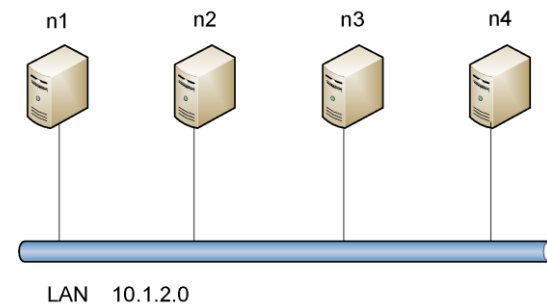
- ✓ P2P Node creation

- ```
NodeContainer p2pNodes;
p2pNodes.Create (2);
```



- ✓ Csm Node creation

- ```
NodeContainer csmaNodes;  
csmaNodes.Add (p2pNodes.Get (1));  
csmaNodes.Create (nCsm);
```



CSMA Simulation Code (1/5)

```
#include "ns3/core-module.h"
#include "ns3/simulator-module.h"
#include "ns3/node-module.h"
#include "ns3/helper-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");

int
main (int argc, char *argv[])
{
    bool verbose = true;
    uint32_t nCsmas = 3;
    CommandLine cmd;
    cmd.AddValue ("nCsmas", "Number of \"extra\" CSMA nodes/devices", nCsmas);
    cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);
    cmd.Parse (argc,argv);
```

CSMA Simulation Code (2/5)

```
if (verbose)
{
    LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
    LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
}
```

```
nCsmma = nCsmma == 0 ? 1 : nCsmma;
```

```
NodeContainer p2pNodes;
p2pNodes.Create (2);
```

```
NodeContainer csmaNodes;
csmaNodes.Add (p2pNodes.Get (1));
csmaNodes.Create (nCsmma);
```

```
PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
```

CSMA Simulation Code (3/5)

```
NetDeviceContainer p2pDevices;  
p2pDevices = pointToPoint.Install (p2pNodes);
```

```
CsmaHelper csma;  
csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));  
csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));
```

```
NetDeviceContainer csmaDevices;  
csmaDevices = csma.Install (csmaNodes);
```

```
InternetStackHelper stack;  
stack.Install (p2pNodes.Get (0));  
stack.Install (csmaNodes);
```

```
Ipv4AddressHelper address;  
address.SetBase ("10.1.1.0", "255.255.255.0");  
Ipv4InterfaceContainer p2pInterfaces;  
p2pInterfaces = address.Assign (p2pDevices);
```

CSMA Simulation Code (4/5)

```
address.SetBase ("10.1.2.0", "255.255.255.0");  
Ipv4InterfaceContainer csmalInterfaces;  
csmalInterfaces = address.Assign (csmadevices);
```

```
UdpEchoServerHelper echoServer (9);
```

```
ApplicationContainer serverApps = echoServer.Install (csmadevices.Get (nCsma));  
serverApps.Start (Seconds (1.0));  
serverApps.Stop (Seconds (10.0));
```

```
UdpEchoClientHelper echoClient (csmalInterfaces.GetAddress (nCsma), 9);  
echoClient.SetAttribute ("MaxPackets", UintegerValue (1));  
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.)));  
echoClient.SetAttribute ("PacketSize", UintegerValue (1024));
```

```
ApplicationContainer clientApps = echoClient.Install (p2pNodes.Get (0));  
clientApps.Start (Seconds (2.0));  
clientApps.Stop (Seconds (10.0));
```


CSMA Simulation Code (5/5)

```
Ipv4GlobalRoutingHelper::PopulateRoutingTables ();  
  
pointToPoint.EnablePcapAll ("second");  
csma.EnablePcap ("second", csmaDevices.Get (1), true);  
  
Simulator::Run ();  
Simulator::Destroy ();  
return 0;  
}
```

Assignment

1. Run mysecond.cc (i.e., ./waf --run..)
2. Modify the **DataRate** of **p2p Channel** (5Mbps -> 1Gbps) by **using command line**
3. Increase the **number of packets** transmitted (**1 -> 3**) by **using command line**
4. Add **two nodes** to the **CSMA Channel**, while maintaining the server's IP in the Example (**using command ./waf --run "scratch/mysecond --nCsma=5"**)
5. Change the **common node** of the **p2p Channel** and the CSMA Channel from n1 to n0 and change the **node sending the data** from n0 to n1

- File Format : HW1_1_your Student ID_your Name.docx (e.g. HW1_20XXXXXXXXX_김기훈.docx)
- Title of the Email : HW1_your Student ID_your Name
- Submit to Email (comnetmnc2023@gmail.com)
- Due date : 10/5 11:59 PM
- **Attach the student number to the capture to prove that you did it yourself**
- **Assignment 1 : Execution result (terminal)**
- **Assignment 2, 3, 4, 5 : Execution result (terminal), code modifications, and brief description of the changed output**

Appendix : Assignment 2

- Using command line arguments can change the simulation parameters on terminal
- Declaration of the **command line parser** in main function is mandatory

```
CommandLine cmd;  
cmd.Parse (argc, argv);
```

- Example 1: **Change the parameters on terminal** (*DataRate, Delay*)

- ✓ First, **DEACTIVATE** the following two lines by adding **//** at the head of the line

```
NodeContainer nodes;  
nodes.Create (2);  
PointToPointHelper pointToPoint;  
//pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));  
//pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));  
NetDeviceContainer devices;  
devices = pointToPoint.Install (nodes);
```

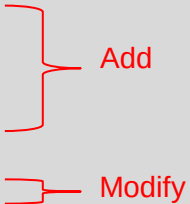
→ In order to use command line arguments, set attribute should be deactivated

- ✓ Then, type the following command
- ✓ ***./waf --run "scratch/my --ns3::PointToPointNetDevice::DataRate=5Mbps --ns3::PointToPointChannel::Delay=2ms"***

Appendix : Assignment 3

- **Creating a new variable and adding a variable as your own hook is possible**
- **Example 2: Create a new variable '*nPackets*' to control the '*MaxPackets*' variable**
 - ✓ When adding your own hooks, use '*cmd.AddValue*' in main function
 - ✓ In '*myfirst.cc*', the **colored lines** must exist

```
int
main (int argc, char *argv[])
{
    uint32_t nPackets =1;
    CommandLine cmd;
    cmd.AddValue("nPackets", "Number of packets to echo", nPackets);
    cmd.Parse (argc, argv);
    ...
    echoClient.SetAttribute ("MaxPackets", UIntegerValue (nPackets));
```

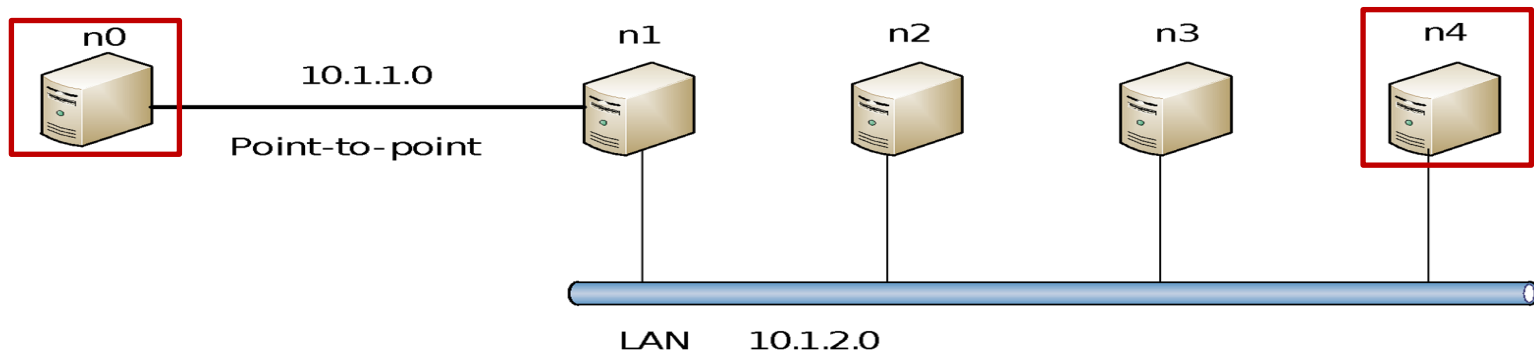


- ✓ Change the parameter *nPacket*
- ✓ *./waf --run "scratch/myfirst --nPackets=2"*

Please type this line with your hand

CSMA Topology

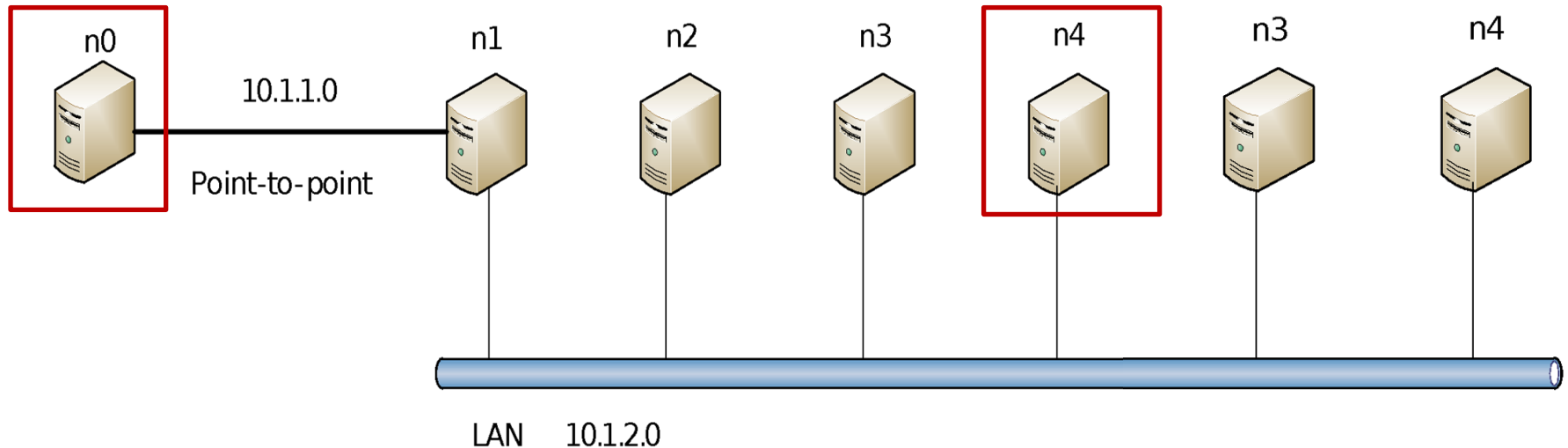
- CSMA network topology examples (*mysecond.cc*)



```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/mysecond
Waf: Entering directory `/home/comnet2023/ns-3.35/build'
Waf: Leaving directory `/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (0.303s)
At time +2s client sent 1024 bytes to 10.1.2.4 port 9
At time +2.0078s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.0078s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.01761s client received 1024 bytes from 10.1.2.4 port 9
```

Appendix : Assignment 4

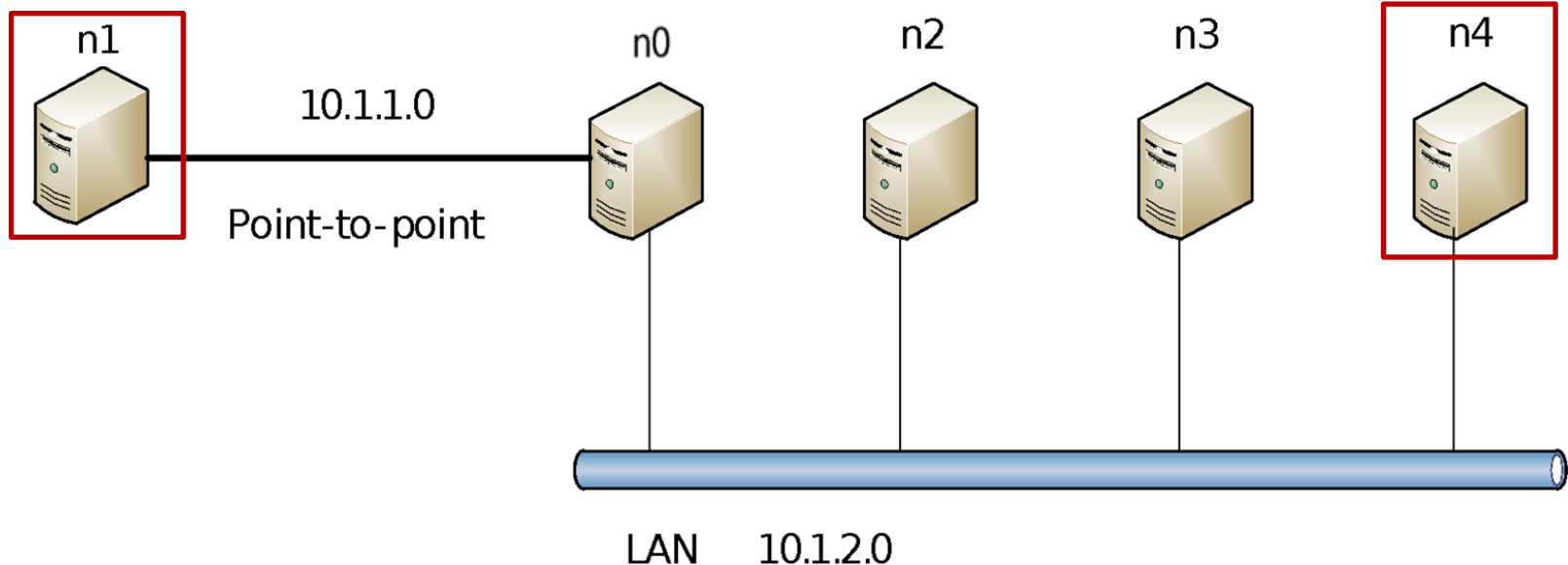
- CSMA network topology examples (*mysecond.cc*)



```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run "scratch/mysecond -nCsma=5"
Waf: Entering directory `/home/comnet2023/ns-3.35/build'
Waf: Leaving directory `/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (0.294s)
At time +2s client sent 1024 bytes to 10.1.2.4 port 9
At time +2.0038s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.0038s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00861s client received 1024 bytes from 10.1.2.4 port 9
```

Appendix : Assignment 5

- CSMA network topology examples (*mysecond.cc*)



```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/mysecond
Waf: Entering directory `/home/comnet2023/ns-3.35/build'
Waf: Leaving directory `/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (0.880s)
At time +2s client sent 1024 bytes to 10.1.2.4 port 9
At time +2.0078s server received 1024 bytes from 10.1.1.2 port 49153
At time +2.0078s server sent 1024 bytes to 10.1.1.2 port 49153
At time +2.01761s client received 1024 bytes from 10.1.2.4 port 9
```

Capture Example

```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/mysecond
Waf: Entering directory `/home/comnet2023/ns-3.35/build'
Waf: Leaving directory `/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (0.299s)
At time +2s client sent 1024 bytes to 10.1.2.4 port 9
At time +2.0078s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.0078s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.01761s client received 1024 bytes from 10.1.2.4 port 9
comnet2023@ubuntu:~/ns-3.35$
```

20xxxxxxxx 김기훈

```
--
34 using namespace ns3;
35
36 NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");
37
38 int
39 main (int argc, char *argv[])
40 {
41     bool verbose = true;
42     uint32_t nCsmas = 3;
43
44     CommandLine cmd (__FILE__);
45     cmd.AddValue ("nCsmas", "Number of \"extra\" CSMA nodes/devices", nCsmas);
46     cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);
47
48     cmd.Parse (argc,argv);
49
50     ...
51 }
```

+

20xxxxxxxx 김기훈